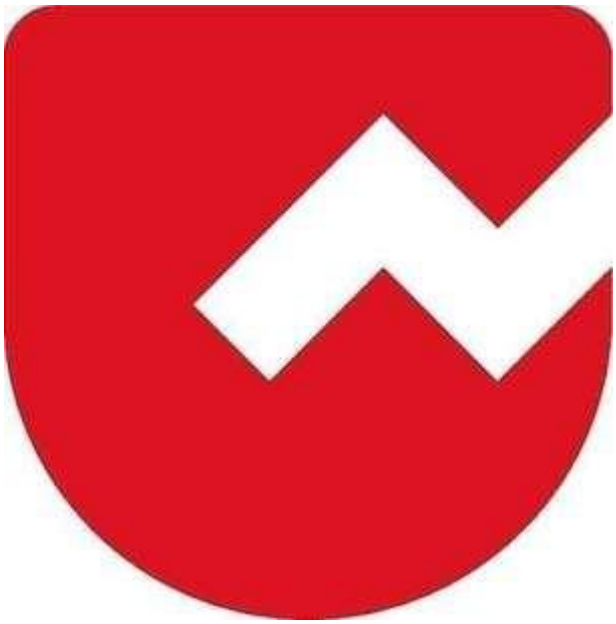
Zoho Schools for Graduate Studies



Notes
Queue

27/11/2025

Understanding the Queue Data Structure

Definition

A Queue is a linear data structure that follows the FIFO (First-In, First-Out) principle. This means the element that has been in the queue the longest is the next one to be removed.

Core Operations

The primary operations for managing elements within a Queue have specific terminology:

- Enqueue: The action of inserting an element into a queue.
- Dequeue: The action of removing or deleting an element from a queue.

Note: The core implementation classes for Queues shown in the Collections table are `PriorityQueue` and `ArrayDeque`.

The time complexity for the fundamental operations of a Queue depends on the specific implementation used. The most common implementations in standard libraries (like Java's `ArrayDeque` or `LinkedList`) are highly optimized.

Here is a breakdown of the time complexity for the core Queue operations:

⏳ Queue Operation Time Complexity

Operation	Implementation	Time Complexity	Notes
Enqueue (Insert at Rear)	Array-based (e.g., `ArrayDeque`)	$O(1)$	Constant time, as the pointer/index to the rear is simply incremented.

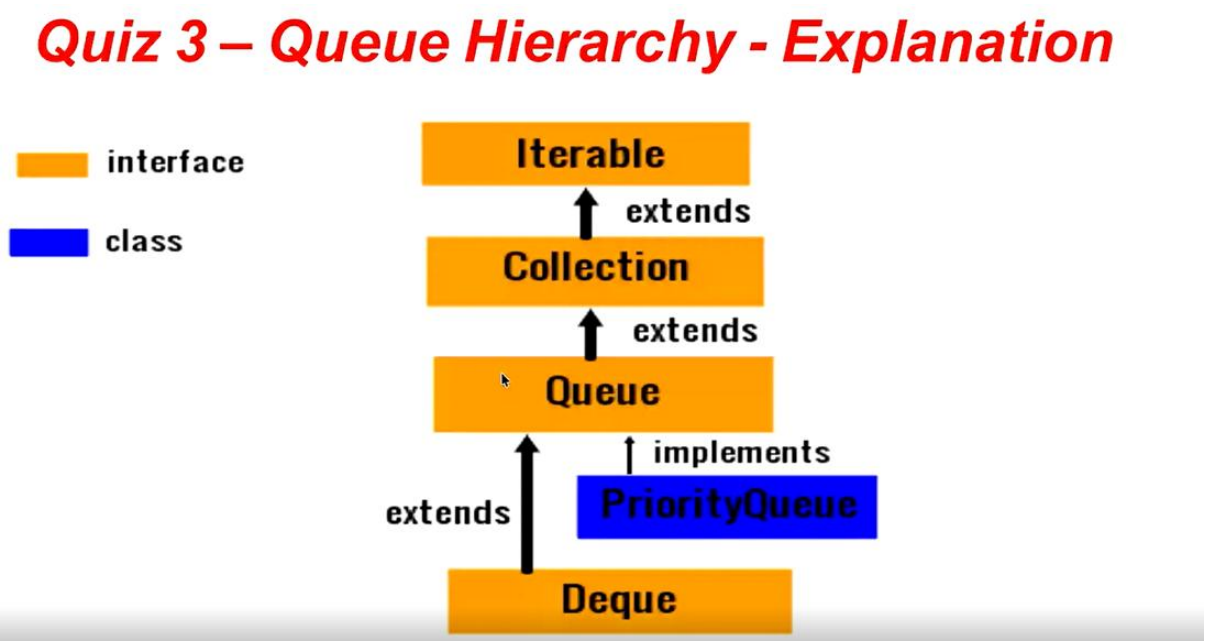
Operation	Implementation	Time Complexity	Notes
	Linked List-based (e.g., LinkedList)	O(1)	Constant time, as a new node is simply attached to the tail of the list.
Dequeue (Remove from Front)	Array-based (e.g., ArrayDeque)	O(1)	Constant time, as the pointer/index to the front is simply incremented.
	Linked List-based (e.g., LinkedList)	O(1)	Constant time, as the head pointer is moved to the next node.
Peek (Get Front Element)	All Implementations	O(1)	Constant time, as it only involves reading the value at the known front index/pointer.

Diffrence in Add & offer method in Queues

Both the add(e) and offer(e) methods are used to insert an element (e) into the Queue. The primary distinction lies in their behavior when the insertion fails, particularly in capacity-restricted (or "bounded") Queue implementations.

Method	Behavior on Successful Insertion	Behavior on Insertion Failure (Queue is Full)	Return Value Type
add(e)	Inserts the element at the tail of the queue.	Throws an IllegalStateException (an exception).	boolean (usually returns true).
offer(e)	Inserts the element at the tail of the queue.	Returns the special value false.	boolean

QUEUE HIERARCHY



Element	Type	Relationship
Iterable	Interface (Orange)	The root of all collections that can be iterated.
Collection	Interface (Orange)	Extends Iterable. The base interface for all collection types (List, Set, Queue).
Queue	Interface (Orange)	Extends Collection. Designed for holding elements prior to processing, usually in a FIFO order.
Deque	Interface (Orange)	Extends Queue. Represents a Double-Ended Queue, allowing element insertion and removal at both ends (front and rear).

Element	Type	Relationship
PriorityQueue	Class (Blue)	Implements Queue. A concrete implementation class that provides priority-based ordering.

⭐ Priority Queue Explanation

A Priority Queue is a specialized extension of the standard Queue data structure, offering priority-based ordering instead of strict FIFO ordering.

Characteristics

- Priority Association: Every element inserted into a priority queue has a priority associated with it.
- Deletion Order: The element with the higher priority will be deleted (removed or polled) *before* elements with lesser priority.

- Tie-Breaker: If two or more elements have the same priority, their relative order is determined by the FIFO (First-In, First-Out) principle.

- Internal Structure: The PriorityQueue class in Java is typically implemented using a Min-Heap, where the element with the highest priority (often the smallest value) is always at the root, making its retrieval $O(1)$.

🛠️ Priority Queue Ordering: Comparable and Iterator

The PriorityQueue class relies on two key mechanisms for element ordering and traversal.

1. Ordering and Comparable / Comparator

The PriorityQueue determines the priority of elements using either the elements' natural ordering or a custom ordering provided at construction time.

- Natural Ordering (via Comparable):
  - If no custom comparator is provided, the elements stored in the PriorityQueue must implement the java.lang.Comparable interface.
  - This interface forces the element class to define its own "natural" comparison logic using the compareTo() method.
  - Example: Standard wrapper classes like Integer and String already implement Comparable.

- Custom Ordering (via Comparator):
  - The PriorityQueue can be constructed with an instance of the java.util.Comparator interface.
  - This is typically used when you need a sorting order different from the natural one, or when the element class is a custom type that doesn't implement Comparable.
  - The Comparator provides the comparison logic externally via the compare(T o1, T o2) method.

2. Traversal and Iterator method

- Implements Iterable: Since PriorityQueue implements the Queue interface, which extends Collection, which extends Iterable, the PriorityQueue provides an iterator() method.
- Iteration Order: The iterator() method does not guarantee traversing the elements in their priority-sorted order. The iterator traverses the elements in the order they are stored in the underlying heap structure (often a level-order traversal), not the sorted logical order.
- For Sorted Traversal: If you require elements in strict priority order, you must repeatedly call the poll() method, which always retrieves and removes the highest-priority element.

Small Quiz on Priority Queue (try on yourself)

```
package part3;
import java.util.*;
public class QueueDemo {
    public static void main(String[] args) {
        PriorityQueue<String>pq=new PriorityQueue<>();
        pq.add("2");pq.add("4");
        System.out.print(pq.peek() + " ");
        pq.offer("1");
        pq.add("3");pq.remove("1");
        System.out.print(pq.poll() + " ");
        if(pq.remove("2"))
            System.out.print(pq.poll() + " ");
        System.out.print(pq.poll() + " " + pq.peek() + " ");

    }
}
```

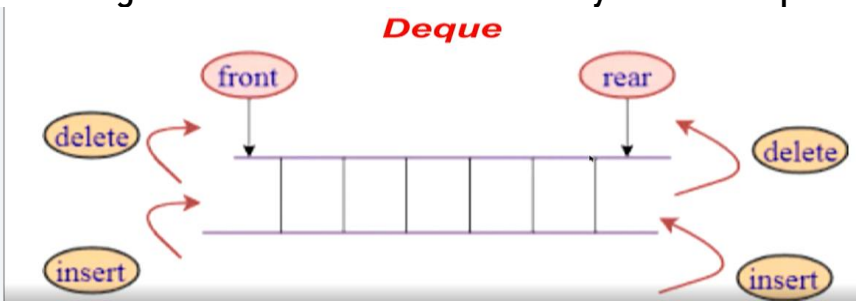
🔄 Deque (Double-Ended Queue) Explanation

A Deque (pronounced "deck" or sometimes "D.E.Q.") stands for Double-Ended Queue. It is a special type of queue that supports the addition and removal of elements from both ends—the front and the rear.

Core Concept

Unlike a standard Queue (which enforces FIFO—First-In, First-Out) or a Stack (which enforces LIFO—Last-In, First-Out), a Deque is more flexible. It provides the functionality of both a queue and a stack in a single data structure, depending on how you choose to use its operations.

The diagram illustrates the flexibility of the Deque:



Key Operations

A Deque allows four primary operations, enabling it to act as a double-ended structure:

Operation	Action	End Affected	Standard Queue/Stack Analog
Insert at Front	Add an element to the beginning.	Front	Stack's Push (LIFO insertion)
Delete from Front	Remove an element from the beginning.	Front	Queue's Dequeue (FIFO removal)
Insert at Rear	Add an element to the end.	Rear	Queue's Enqueue (FIFO insertion)
Delete from Rear	Remove an element from the end.	Rear	Stack's Pop (LIFO removal)

Implementations in Java

In the Java Collections Framework, the `Deque` is an interface that extends the `Queue` interface. Common concrete classes that implement `Deque` include:

1. ArrayDeque: An array-based implementation that resizes dynamically (like ArrayList). It is generally the preferred and most efficient implementation for Deques and Queues in Java.
2. LinkedList: A doubly-linked list implementation. While it supports the Deque interface, ArrayDeque is often faster for most Deque operations.

Time Complexity

The major advantage of well-implemented Deques (like ArrayDeque) is that all core insertion and deletion operations at both the front and rear are performed in constant time, O(1)

Array Deque

- Unlike Queue, one can add or remove elements from both sides.
- Null elements are not allowed.
- It is not thread safe, in the absence of external synchronization.
- It has no capacity restrictions.
- It is faster than LinkedList and Stack.

Expected Output:

```
import java.util.ArrayDeque;
class Main {
    public static void main(String[] args) {
        ArrayDeque<String> ad = new ArrayDeque<>();
        ad.add("2");
        ad.add("4");
        System.out.print(ad.peek() + ", ");
        ad.offer("1");
        ad.add("3");
        ad.remove();
        System.out.print(ad.peek() + ", ");
        If(ad.peek().equals("2")){
                System.out.println(ad.poll() + " ");
        System.out.print(ad.poll() + " " + ad.peek() + " ");
    }
}
```

Output:

2 4 1 3